

PEMROGRAMAN BERORIENTASI OBJEK

PRAKTIKUM

NAMA : IDHAM KHOLID NUR AZIZI

NIM : 20230801192

CR002, CSF308

1. LAPORAN PROGRAM CLASS MAHASISWA

```
package tugas;

public class Mahasiswa {
    public String nama;        // Public: bisa diakses dari mana saja
    protected int usia;       // Protected: bisa diakses dari dalam package
    dan subclass
    private String jurusan;    // Private: hanya bisa diakses dalam kelas ini

    // Constructor
    public Mahasiswa(String nama, int usia, String jurusan) {
        this.nama = nama;
        this.usia = usia;
        this.jurusan = jurusan;
    }

    // Getter untuk atribut private jurusan
    public String getJurusan() {
        return jurusan;
    }

    // Setter untuk atribut private jurusan
    public void setJurusan(String jurusanBaru) {
        this.jurusan = jurusanBaru;
    }

    // Metode untuk menampilkan informasi
    public void tampilkanInfo() {
```

```

        System.out.println("Nama: " + nama);
        System.out.println("Usia: " + usia);
        System.out.println("Jurusan: " + jurusan);
    }
}

// Kelas utama untuk menjalankan program
class MhsTester {
    public static void main(String[] args) {
        // Membuat objek Mahasiswa
        Mahasiswa mahasiswa1 = new Mahasiswa("Andi", 21, "Teknik
Informatika");

        // Mengakses atribut public
        System.out.println("Nama Mahasiswa: " + mahasiswa1.nama); // Output:
Andi

        // Mengakses atribut protected (dapat diakses dalam package yang sama)
        System.out.println("Usia Mahasiswa: " + mahasiswa1.usia); // Output:
21

        // Mengakses atribut private menggunakan getter
        System.out.println("Jurusan Mahasiswa: " +
mahasiswa1.getJurusan()); // Output: Teknik Informatika

        // Mengubah nilai atribut private menggunakan setter
        mahasiswa1.setJurusan("Sistem Informasi");
        System.out.println("Jurusan Mahasiswa Setelah Diubah: " +
mahasiswa1.getJurusan()); // Output: Sistem Informasi

        // Menampilkan informasi lengkap mahasiswa
        mahasiswa1.tampilkanInfo();
    }
}

```

1) Analisis Akses Atribut dan Metode

- Mengakses atribut public: mahasiswa1.nama
 - Karena nama adalah public, atribut ini dapat diakses dan ditampilkan langsung.
- Mengakses atribut protected: mahasiswa1.usia
 - Karena usia adalah protected, atribut ini juga bisa diakses dalam kelas MhsTester, yang berada dalam package yang sama.
- Mengakses atribut private menggunakan getter: mahasiswa1.getJurusan()
 - Karena jurusan adalah private, maka atribut ini tidak dapat diakses langsung. Pengaksesan jurusan dilakukan dengan menggunakan getter, yaitu metode getJurusan().
- Mengubah nilai atribut private menggunakan setter: mahasiswa1.setJurusan("Sistem Informasi")

- Untuk mengubah nilai jurusan, digunakan setter setJurusan, yang memungkinkan perubahan nilai jurusan dari luar kelas Mahasiswa.

2) Class Mahasiswa

- Atribut
 - nama: Atribut dengan akses modifier public, sehingga bisa diakses dari mana saja, termasuk dari luar kelas atau package.
 - usia: Atribut dengan akses modifier protected, sehingga hanya bisa diakses dari kelas lain dalam package yang sama atau dari subclass-nya.
 - jurusan: Atribut dengan akses modifier private, sehingga hanya bisa diakses dari dalam kelas Mahasiswa sendiri. Untuk mengakses atau mengubah nilai jurusan, kita membutuhkan metode getter dan setter.
- Constructor:
 - Constructor Mahasiswa(String nama, int usia, String jurusan) digunakan untuk menginisialisasi objek Mahasiswa dengan nilai nama, usia, dan jurusan.
- Getter dan Setter:
 - Metode getJurusan(): Merupakan metode getter yang memungkinkan pengaksesan atribut private jurusan dari luar kelas.
 - Metode setJurusan(String jurusanBaru): Merupakan metode setter yang memungkinkan perubahan nilai private jurusan dari luar kelas.
- Metode tampilkanInfo():
 - Metode ini berfungsi untuk menampilkan semua informasi dari objek Mahasiswa, termasuk nama, usia, dan jurusan.

3) Kelas MhsTester

- Kelas MhsTester berfungsi sebagai kelas utama untuk menjalankan program.
- Di dalam main, terdapat proses pembuatan objek Mahasiswa bernama mahasiswa1 dengan data nama = "Andi", usia = 21, dan jurusan = "Teknik Informatika".

4) Output Program

```

TERMINAL
PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Cod e\User\workspaceStorage\7c4583c3383b58d273aa678e1bb04018\redhat.java\jdt_ws\jdt.ls-java-project\bin' 'tugas.MhsTester'
Nama Mahasiswa: Andi
Usia Mahasiswa: 21
Jurusan Mahasiswa: Teknik Informatika
Jurusan Mahasiswa Setelah Diubah: Sistem Informasi
Nama: Andi
Usia: 21
Jurusan: Sistem Informasi
PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002>

```

2. LAPORAN PROGRAM MOBIL

```
package main.java.tugas;

class Mobil {
    public String merk;           // Public: bisa diakses dari mana
    saja
    protected int tahunProduksi; // Protected: bisa diakses dari
    dalam package dan subclass
    private double harga;        // Private: hanya bisa diakses dalam
    kelas ini

    // Constructor
    public Mobil(String merk, int tahunProduksi, double harga) {
        this.merk = merk;
        this.tahunProduksi = tahunProduksi;
        this.harga = harga;
    }

    // Getter untuk atribut private harga
    public double getHarga() {
        return harga;
    }

    // Setter untuk atribut private harga
    public void setHarga(double hargaBaru) {
        if (hargaBaru > 0) {      // Validasi harga baru harus
positif
            this.harga = hargaBaru;
        } else {
            System.out.println("Harga harus lebih besar dari 0.");
        }
    }

    // Metode untuk menampilkan informasi mobil
    public void tampilkanInfoMobil() {
        System.out.println("Merk: " + merk);
        System.out.println("Tahun Produksi: " + tahunProduksi);
        System.out.println("Harga: " + harga);
    }
}

// Kelas utama untuk menjalankan program
public class MobilCek {
    public static void main(String[] args) {
        // Membuat objek Mobil
        Mobil mobil1 = new Mobil("Toyota", 2022, 300000000);

        // Mengakses atribut public
    }
}
```

```

        System.out.println("Merk Mobil: " + mobil1.merk); //
Output: Toyota

        // Mengakses atribut protected (dapat diakses dalam package yang sama)
        System.out.println("Tahun Produksi Mobil: " + mobil1.tahunProduksi);
// Output: 2022

        // Mengakses atribut private menggunakan getter
        System.out.println("Harga Mobil: " + mobil1.getHarga()); //
Output: 300000000.0

        // Mengubah nilai atribut private menggunakan setter
        mobil1.setHarga(350000000);
        System.out.println("Harga Mobil Setelah Diubah: " +
mobil1.getHarga()); // Output: 350000000.0

        // Menampilkan informasi lengkap mobil
        mobil1.tampilkanInfoMobil();
    }
}

```

1) Analisis Penggunaan Atribut dan Metode

- Mengakses atribut public: mobil1.merk
 - Karena merk memiliki akses public, atribut ini dapat diakses dan ditampilkan langsung dari luar kelas.
- Mengakses atribut protected: mobil1.tahunProduksi
 - Karena tahunProduksi memiliki akses protected, atribut ini juga dapat diakses dari kelas MobilCek yang berada dalam package yang sama.
- Mengakses atribut private menggunakan getter: mobil1.getHarga()
 - Karena harga memiliki akses private, atribut ini tidak dapat diakses langsung. Pengaksesan harga dilakukan menggunakan getter, yaitu metode getHarga().
- Mengubah nilai atribut private menggunakan setter: mobil1.setHarga(350000000)
 - Untuk mengubah nilai harga, digunakan setter setHarga(). Dengan ini, kita dapat mengubah nilai harga dari luar kelas Mobil, asalkan nilai yang dimasukkan positif (sesuai dengan validasi pada metode setHarga).

2) Output Program

```

▼ TERMINAL
PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002> & 'C:\Program
Files\Java\jdk-23\bin\java.exe' '-enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Cod
e\User\workspaceStorage\7c4583c3383b58d273aa678e1bb04018\redhat.java\jdt_ws\jdt.ls-java-project\bin' 'main.java.tugas.MobilCek'
Merk Mobil: Toyota
Tahun Produksi Mobil: 2022
Harga Mobil: 3.0E8
Harga Mobil Setelah Diubah: 3.5E8
Merk: Toyota
Tahun Produksi: 2022
Harga: 3.5E8
PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002>

```

3. LAPORAN PROGRAM NilaiTester

```
package tugas;

public class NilaiTester {

    static class Nilai {
        private double quis;
        private double uts;
        private double uas;

        // Setter untuk quis dengan validasi nilai antara 0 dan 100
        public void setQuis(double x) {
            if (x >= 0 && x <= 100) {
                quis = x;
            } else {
                System.out.println("Nilai quis harus antara 0 dan 100.");
            }
        }

        // Setter untuk uts dengan validasi nilai antara 0 dan 100
        public void setUTS(double x) {
            if (x >= 0 && x <= 100) {
                uts = x;
            } else {
                System.out.println("Nilai UTS harus antara 0 dan 100.");
            }
        }

        // Setter untuk uas dengan validasi nilai antara 0 dan 100
        public void setUAS(double x) {
            if (x >= 0 && x <= 100) {
                uas = x;
            } else {
                System.out.println("Nilai UAS harus antara 0 dan 100.");
            }
        }

        // Getter untuk quis
        public double getQuis() {
            return quis;
        }

        // Getter untuk uts
        public double getUTS() {
            return uts;
        }

        // Getter untuk uas
    }
}
```

```

    public double getUAS() {
        return uas;
    }

    // Menghitung nilai akhir (NA)
    public double getNA() {
        return 0.20 * quis + 0.30 * uts + 0.50 * uas;
    }
}

public static void main(String[] args) {
    Nilai n = new Nilai();

    // Mengisi nilai quis, uts, dan uas
    n.setQuis(90);
    n.setUTS(70);
    n.setUAS(150); // Nilai tidak valid, akan menampilkan pesan kesalahan

    // Menampilkan nilai akhir
    System.out.println("Nilai Akhir (NA): " + n.getNA());
}
}

```

1) Analisis

- Enkapsulasi:
 - Kelas Nilai menjaga atribut quis, uts, dan uas dengan akses private, sehingga data tetap aman dan hanya dapat diubah melalui metode setter.
- Validasi Data:
 - Metode setQuis, setUTS, dan setUAS memiliki validasi rentang nilai antara 0 dan 100. Ini memastikan data yang diterima valid sebelum disimpan dalam atribut.
- Pengelolaan Nilai Akhir:
 - Metode getNA menggunakan bobot yang berbeda untuk quis, uts, dan uas, yang menunjukkan perhitungan nilai akhir yang realistis.

2) Class Nilai

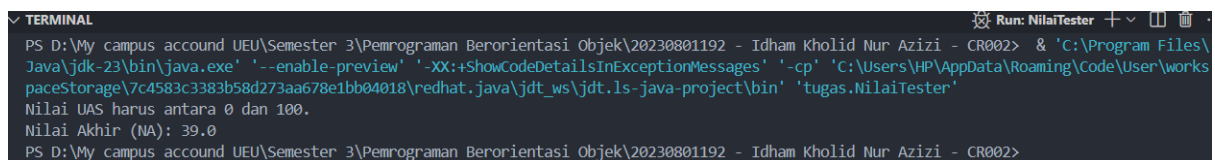
- Kelas Nilai adalah inner class dari NilaiTester, berisi atribut dan metode untuk mengelola nilai quis, uts, dan uas.
- Atribut:
 - quis, uts, dan uas: Ketiga atribut ini memiliki akses modifier private, sehingga hanya bisa diakses dari dalam kelas Nilai. Akses ini mengikuti prinsip enkapsulasi untuk melindungi data dan menjamin integritas nilai.
- Setter dengan Validasi:
 - setQuis(double x): Metode ini berfungsi untuk menetapkan nilai quis dengan validasi bahwa nilai harus berada dalam rentang 0 hingga 100. Jika nilai di luar rentang tersebut, akan muncul pesan kesalahan: "Nilai quis harus antara 0 dan 100."

- setUTS(double x) dan setUAS(double x): Keduanya berfungsi mirip dengan setQuis, memastikan nilai uts dan uas berada dalam rentang 0-100. Jika tidak, masing-masing akan menampilkan pesan kesalahan yang relevan.
- Getter:
 - getQuis(), getUTS(), dan getUAS() adalah metode getter untuk mengakses nilai quis, uts, dan uas dari luar kelas Nilai, karena atributnya memiliki akses private.

3) Class NilaiTester

- Di dalam main, dibuat objek Nilai bernama n.
- Mengisi Nilai quis, uts, dan uas:
 - n.setQuis(90): Nilai quis diatur ke 90, yang valid (berada dalam rentang 0-100).
 - n.setUTS(70): Nilai uts diatur ke 70, yang valid.
 - n.setUAS(150): Nilai uas diatur ke 150, yang tidak valid. Karena nilai ini berada di luar rentang 0-100, maka akan muncul pesan kesalahan: "Nilai UAS harus antara 0 dan 100." Nilai uas tetap berada pada nilai default (yakni 0), karena perubahan tidak dilakukan.

4) Output Program



```

PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\7c4583c3383b58d273aa678e1bb04018\redhat.java\jdt_ws\jdt.ls-java-project\bin' 'tugas.NilaiTester'
Nilai UAS harus antara 0 dan 100.
Nilai Akhir (NA): 39.0
PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002>
  
```

4. LAPORAN PROGRAM SISWA

```

package tugas;

class Siswa {
    private String nama;          // Private: hanya bisa diakses dalam kelas ini
    private int nilaiUjian;       // Private: hanya bisa diakses dalam kelas ini

    // Constructor
    public Siswa(String nama, int nilaiUjian) {
        this.nama = nama;
        this.nilaiUjian = nilaiUjian; // Set nilai langsung di konstruktor tanpa setter
    }

    // Getter untuk nama
    public String getNama() {
        return nama;
    }

    // Setter untuk nama
    public void setNama(String nama) {
  
```



```

        this.nama = nama;
    }

    // Getter untuk nilai ujian
    public int getNilaiUjian() {
        return nilaiUjian;
    }

    // Setter untuk nilai ujian dengan validasi
    public void setNilaiUjian(int nilaiUjian) {
        // Validasi nilai (0-100)
        if (nilaiUjian >= 0 && nilaiUjian <= 100) {
            this.nilaiUjian = nilaiUjian;
        } else {
            System.out.println("Nilai harus antara 0 dan 100.");
        }
    }

    // Metode untuk menampilkan informasi siswa
    public void tampilkanInfo() {
        System.out.println("Nama Siswa: " + nama);
        System.out.println("Nilai Ujian: " + nilaiUjian);
    }
}

// Kelas utama untuk menjalankan program
public class SiswaTester {
    public static void main(String[] args) {
        // Membuat objek Siswa
        Siswa siswa1 = new Siswa("Andi", 85);
        siswa1.tampilkanInfo();

        // Menggunakan setter untuk mengubah nama dan nilai ujian
        siswa1.setNama("Budi");
        siswa1.setNilaiUjian(95);

        // Menampilkan informasi yang telah diperbarui
        System.out.println("\nSetelah Diubah:");
        siswa1.tampilkanInfo();

        // Menguji validasi dengan memasukkan nilai yang tidak valid
        siswa1.setNilaiUjian(105); // Output: Nilai harus antara 0 dan 100.
    }
}

```

- 1) Class Siswa
 - Atribut:

- nama dan nilaiUjian: Keduanya memiliki akses modifier private, sehingga hanya dapat diakses dari dalam kelas Siswa. Hal ini mengikuti prinsip enkapsulasi untuk menjaga agar data hanya bisa diubah melalui metode yang telah ditentukan (getter dan setter).
- Constructor:
 - Constructor Siswa(String nama, int nilaiUjian) digunakan untuk menginisialisasi objek Siswa dengan nama dan nilaiUjian. Nilai ujian disetel langsung dalam konstruktor tanpa menggunakan metode setter, karena pada saat pembuatan objek biasanya nilai sudah dianggap valid.
- Getter dan Setter:
 - Getter getNama() dan getNilaiUjian(): Digunakan untuk mendapatkan nilai atribut nama dan nilaiUjian dari luar kelas Siswa.
 - Setter setNama(String nama): Digunakan untuk mengubah nama siswa dari luar kelas Siswa.
 - Setter setNilaiUjian(int nilaiUjian): Digunakan untuk mengubah nilai ujian dengan validasi bahwa nilai harus berada dalam rentang 0 hingga 100. Jika nilai yang dimasukkan tidak valid (di luar rentang tersebut), akan menampilkan pesan kesalahan: "Nilai harus antara 0 dan 100." Jika nilai tidak valid, nilaiUjian tidak akan diubah.

2) Class SiswaTester

- Membuat Objek Siswa:
 - Di dalam main, dibuat objek Siswa bernama siswa1 dengan data nama = "Andi" dan nilaiUjian = 85. Metode tampilkanInfo() digunakan untuk menampilkan informasi awal siswa tersebut.
- Menggunakan Setter untuk Mengubah Data:
 - siswa1.setNama("Budi"): Nama siswa diubah menjadi "Budi" menggunakan setter.
 - siswa1.setNilaiUjian(95): Nilai ujian siswa diubah menjadi 95 menggunakan setter. Setelah perubahan, tampilkanInfo() dipanggil lagi untuk memastikan data yang diperbarui ditampilkan.
- Menguji Validasi dengan Nilai yang Tidak Valid:
 - siswa1.setNilaiUjian(105): Dicoba memberikan nilai ujian yang tidak valid, yaitu 105, yang berada di luar rentang 0-100. Karena nilai ini tidak valid, program akan menampilkan pesan kesalahan: "Nilai harus antara 0 dan 100." Nilai nilaiUjian tetap tidak berubah.

3) Output Program

```

TERMINAL
PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Cod e\User\workspaceStorage\7c4583c3383b58d273aa678e1bb04018\redhat.java\jdt_ws\jdt.ls-java-project\bin' 'tugas.SiswaTester'
Nama Siswa: Andi
Nilai Ujian: 85

Setelah Diubah:
Nama Siswa: Budi
Nilai Ujian: 95
Nilai harus antara 0 dan 100.
PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002>

```

5. LAPORAN PROGRAM WAKTU

```
package tugas;

public class WaktuTester {

    static class Waktu {
        private int menitWaktu; // Menyimpan total waktu dalam menit

        // Constructor
        public Waktu() {
            this.menitWaktu = 0;
        }

        // Menambahkan jam ke waktu (j * 60 menit)
        public void tambahJam(int j) {
            this.menitWaktu += j * 60;
        }

        // Menambahkan menit ke waktu
        public void tambahMenit(int m) {
            this.menitWaktu += m;
        }

        // Menambahkan jam dan menit ke waktu
        public void tambahWaktu(int j, int m) {
            this.menitWaktu += (j * 60) + m;
        }

        // Menampilkan waktu dalam format jam dan menit
        public void tampilWaktu() {
            int jam = menitWaktu / 60;
            int menit = menitWaktu % 60;
            System.out.println(jam + " jam " + menit + " menit");
        }
    }

    public static void main(String[] args) {
        // Membuat objek Waktu pertama dan menambah waktu
        Waktu waktu1 = new Waktu();
        waktu1.tambahWaktu(2, 30); // Menambahkan 2 jam 30 menit
        waktu1.tampilWaktu();      // Output: 2 jam 30 menit

        // Membuat objek Waktu kedua dan menambah waktu
        Waktu waktu2 = new Waktu();
        waktu2.tambahWaktu(3, 45); // Menambahkan 3 jam 45 menit
        waktu2.tampilWaktu();      // Output: 3 jam 45 menit
    }
}
```

1) Class Waktu

- Kelas Waktu berfungsi untuk menyimpan dan mengelola waktu dalam bentuk menit, dengan metode untuk menambah dan menampilkan waktu dalam format jam dan menit.
- Atribut:
 - menitWaktu: Atribut ini bertipe int dan memiliki akses private, digunakan untuk menyimpan total waktu dalam satuan menit. Akses private melindungi data agar hanya dapat diubah melalui metode yang ada dalam kelas ini.
- Constructor:
 - Constructor Waktu() menginisialisasi objek Waktu dengan menitWaktu bernilai 0. Ini berarti objek Waktu baru akan dimulai dari waktu 0 jam 0 menit.
- Metode:
 - tambahJam(int j): Menambahkan jam ke menitWaktu. Setiap jam dikonversi ke menit dengan perkalian $j * 60$, lalu ditambahkan ke menitWaktu.
 - tambahMenit(int m): Menambahkan menit langsung ke menitWaktu.
 - tambahWaktu(int j, int m): Menambahkan waktu dalam jam dan menit secara bersamaan. Jam dikonversi ke menit ($j * 60$), kemudian ditambah dengan menit, dan hasilnya ditambahkan ke menitWaktu.
 - tampilWaktu(): Menampilkan waktu dalam format jam dan menit dengan mengonversi menitWaktu menjadi jam dan menit.
 - Jam dihitung sebagai $\text{menitWaktu} / 60$, dan sisa menit dihitung dengan $\text{menitWaktu} \% 60$.

2) Class WaktuTester

- Membuat Objek Waktu:
 - Di dalam main, dibuat dua objek Waktu, yaitu waktu1 dan waktu2.
- Penggunaan tambahWaktu() untuk Menambah Waktu:
- waktu1.tambahWaktu(2, 30): Metode tambahWaktu menambahkan 2 jam dan 30 menit ke objek waktu1, menghasilkan total waktu 2 jam 30 menit.
- waktu1.tampilWaktu(): Menampilkan total waktu yang tersimpan dalam waktu1, yaitu "2 jam 30 menit".
- waktu2.tambahWaktu(3, 45): Menambahkan 3 jam dan 45 menit ke objek waktu2, menghasilkan total waktu 3 jam 45 menit.
- waktu2.tampilWaktu(): Menampilkan total waktu yang tersimpan dalam waktu2, yaitu "3 jam 45 menit".

3) Output Program

```
▼ TERMINAL Run: WaktuTester + - □ ×
PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\7c4583c3383b58d273aa678e1bb04018\redhat.java\jdt_ws\jdt.ls-java-project\bin' 'tugas.waktuTester'
2 jam 30 menit
3 jam 45 menit
PS D:\My campus account UEU\Semester 3\Pemrograman Berorientasi Objek\20230801192 - Idham Kholid Nur Azizi - CR002>
```